GitHub Commands – Deep Explanation Notes

---

1. What is Git and GitHub?

Git

Git is a **Version Control System**.

It helps developers:

- Save code history
- Track changes
- Work in teams
- Restore old code
- Manage versions

GitHub

GitHub is a **cloud platform** where Git repositories are stored online.

Think:

- **Git = Tool**
- **GitHub = Website using Git**

Example:

You write code in laptop → Save using Git → Upload to GitHub.

---

2. Why Learn GitHub Commands

Because in companies you will:

- Push code daily
- Work in teams
- Create branches
- Fix conflicts
- Pull latest code
- Review others code

---

3. Important Basic Terms

Word	Meaning
Repository	Project folder
Commit	Save point
Branch	Separate line of work
Push	Upload code to GitHub
Pull	Download latest code
Clone	Copy project from GitHub
Merge	Combine branches

---

4. Installation Check

```
git --version
```

Meaning:

Checks Git installed or not.

---

5. First Time Setup

Set Username

```
git config --global user.name "Akshat"
```

Set Email

```
git config --global user.email "youremail@gmail.com"
```

Why?

Every commit stores:

- Your name
- Your email

---

6. Start New Project

Initialize Git

```
git init
```

Meaning:

Converts normal folder into Git repository.

After this hidden `.git` folder created.

This folder stores history.

---

7. Check Status (Most Important Command)

```
git status
```

Use:

Shows:

- New files
- Modified files
- Staged files
- Commit pending

Example:

```
modified: index.js
```

Means file changed.

---

8. Add Files to Staging Area

Add One File

```
git add index.js
```

Add All Files

```
git add .
```

Meaning:

Prepare files for commit.

Think:

Work area → Staging area → Commit

---

9. Commit Changes

```
git commit -m "Added login page"
```

Meaning:

Creates permanent save point.

`-m`

Means message.

Good messages:

```
git commit -m "Fixed navbar bug"
git commit -m "Added payment API"
```

Bad:

```
git commit -m "done"
```

---

10. View History

```
git log
```

Shows:

- Commit ID
- Author
- Date
- Message

---

11. Short History

```
git log --oneline
```

Better for students.

Example:

```
a45bc Added login page
b76cd Fixed css
```

---

12. Connect Local Project to GitHub

```
git remote add origin https://github.com/username/project.git
```

Meaning:

Connect local folder to GitHub repository.

Breakdown:

- `remote` = online link
- `origin` = default name
- URL = repository link

---

13. Push Code to GitHub

```
git push -u origin main
```

Meaning:

Upload code.

Breakdown:

- `push` = send code
- `origin` = GitHub repo
- `main` = branch name

`-u`

Sets default tracking.

Next time:

```
git push
```

Enough.

---

14. Clone Existing Repository

```
git clone https://github.com/username/project.git
```

Meaning:

Download project from GitHub.

Creates folder automatically.

---

15. Pull Latest Code

```
git pull origin main
```

Meaning:

Download latest changes + merge.

Use daily in company.

---

16. Branch Commands

See Branches

```
git branch
```

Current branch has `*`

Create Branch

```
git branch feature-login
```

Switch Branch

```
git checkout feature-login
```

Modern way:

```
git switch feature-login
```

Create + Switch Together

```
git checkout -b feature-login
```

or

```
git switch -c feature-login
```

Why Branch Used?

Suppose login feature working separately.

Main project safe.

---

17. Merge Branch

Go main branch:

```
git checkout main
```

Then:

```
git merge feature-login
```

Meaning:

Combine login branch into main.

---

18. Delete Branch

```
git branch -d feature-login
```

After merge usually delete.

---

19. Remove File from Git

```
git rm file.txt
```

Deletes from folder + Git tracking.

---

20. Rename File

```
git mv old.js new.js
```

---

21. Undo Commands

Unstage File

```
git reset index.js
```

Removes from staging.

Undo Last Commit (keep code)

```
git reset --soft HEAD~1
```

Undo Last Commit (remove staging)

```
git reset --mixed HEAD~1
```

Delete Changes

```
git checkout -- index.js
```

Restores old file.

---

22. See Difference

```
git diff
```

Shows line changes.

Red = removed
Green = added

---

23. .gitignore File

Used to ignore files.

Create:

```
.gitignore
```

Example:

```
node_modules/
.env
dist/
```

Why?

Do not upload:

- node_modules
- passwords
- build files

---

24. Real Company Workflow

```
git pull
git checkout -b feature-dashboard
# code changes
git add .
git commit -m "Created dashboard UI"
git push origin feature-dashboard
```

Then Pull Request on GitHub.

---

25. Daily Student Workflow

```
git status
git add .
git commit -m "Today practice completed"
git push
```

---

26. Git Areas (Very Important)

```
Working Directory
    ↓
Staging Area
    ↓
Repository
```

Example:

- Write code → Working area
- `git add .` → Staging
- `git commit` → Saved history

---

27. Most Asked Interview Commands

Command	Purpose
git init	Start git
git clone	Copy repo

Command	Purpose
git status	Check files
git add .	Stage files
git commit -m	Save changes
git push	Upload
git pull	Download
git branch	Show branches
git checkout	Switch branch
git merge	Merge code

---

28. Common Errors

Error:

```
fatal: not a git repository
```

Fix:

```
git init
```

Error:

```
rejected push
```

Fix:

```
git pull origin main
git push
```

---

29. Best Commit Message Rules

Use action words:

```
Added login page
Fixed sidebar issue
Updated API logic
Removed unused code
```

---

30. Shortcut Revision

```
git init
git status
git add .
git commit -m "message"
git push
git pull
git branch
git checkout branchname
git merge branchname
```

---

31. Final Memory Trick

```
Write Code
Add
Commit
Push
```

Full Flow:

```
Create → Add → Commit → Push
```

---

32. Explain Like This

Git is like **game save points**.

- Commit = Save game
- Branch = New mission
- Merge = Mission complete join main story
- GitHub = Cloud backup

---

33. Homework Practice

Create project and run:

```
git init
git add .
git commit -m "First commit"
git branch feature-page
git checkout feature-page
git commit -m "Added page"
git checkout main
git merge feature-page
git push
```

---

34. Pro Tip

Use everyday for every project.

Even calculator project.

Then commands become natural.